UNIVERSIDAD DE MANIZALES
FACULTAD DE CIENCIAS E INGENIERÍAS
INGENIERÍA EN ANALÍTICA DE DATOS

DOCUMENTO DE DECISIONES DE DISEÑO
Actividad Colaborativa N°2: Evolución del Sistema de Gestión Académica

Integrantes:
Legna Liliana Ferreira Rosas
Juan David Perdomo Rojas

Asignatura:
Programación I

Manizales, Colombia
octubre de 2026

VIGILADO Por el Ministerio de Educación Nacional
Campusvirtual Universidad de Manizales
ISO 9001:2008 MSC-57117129 www.fbcert.services www.certidata.info
Acreditación Institucional de Alta Calidad Resolución 4792 del 15 de mayo de 2019
UNIVERSIDAD DE MANIZALES

1. Introducción

El desarrollo de software orientado a objetos permite modelar dominios reales mediante entidades que encapsulan estado y comportamiento, evolucionando de manera incremental a medida que surgen nuevos requerimientos institucionales. En la primera versión del Sistema de Gestión Académica (que hemos realizado en la Actividad Colaborativa N°1), se establecieron los cimientos del modelo universitario mediante las clases Persona, Estudiante, Docente, Administrativo, ProgramaAcademico y Asignatura, aplicando encapsulamiento, herencia, sobreescritura de métodos y polimorfismo.

El presente Documento de Decisiones de Diseño expone y justifica técnicamente las decisiones arquitectónicas adoptadas durante la Actividad Colaborativa N° 2, cuyo propósito es evolucionar el mismo sistema para agregar la oferta académica (grupos y horarios) y la gestión del proceso de matrícula estudiantil. A lo largo del documento se explica la transformación de la clase Persona en una clase abstracta, la implementación polimórfica de los comportamientos en sus subclases, el diseño de las relaciones de asociación, agregación y composición entre GrupoAcademico, Asignatura, Docente, Estudiante y Horario, así como la implementación de una jerarquía de excepciones personalizadas que garantizan la continuidad de la ejecución del programa.

2. Descripción General de la Solución Implementada

La solución fue construida de manera evolutiva sobre la base de código de la versión v1.0, localizada en el repositorio de GitHub, preservando el sistema como es indicado en los criterios de evaluación y evitando crear programas independientes por fase. La arquitectura actualizada del proyecto se organiza en diez módulos en Python:

- **persona.py (Clase Persona):** Clase base abstracta (hereda de ABC) que define los atributos comunes (__identificacion, __nombre, __correo), el método concreto mostrar_informacion() y el método abstracto realizar_actividad_principal().
- **estudiante.py (Clase Estudiante):** Subclase concreta de Persona que añade __codigo_estudiantil, __programa (instancia de ProgramaAcademico), __semestre y __promedio.
- **docente.py (Clase Docente):** Subclase concreta de Persona que incorpora __numero_empleado, __facultad, __tipo_contratacion y __horas.
- **administrativo.py (Clase Administrativo):** Subclase concreta de Persona que gestiona __numero_empleado, __dependencia, __cargo y __jornada.
- **programa_academico.py (Clase ProgramaAcademico):** Representa los programas académicos ofertados e incorpora el método estático buscar_programa(lista_programas, codigo) con control de excepciones.
- **asignatura.py (Clase Asignatura):** Representa las asignaturas asociadas a un ProgramaAcademico e incluye el método estático buscar_asignatura(lista_asignaturas, codigo).
- **horario.py (Clase Horario):** Modela la programación temporal y física de un grupo mediante los atributos privados __dia, __hora_inicio, __hora_finalizacion y __salon.

VIGILADO Por el Ministerio de Educación Nacional
Campusvirtual Universidad de Manizales
ISO 9001:2008 MSC-57117129 www.fbcert.services www.certidata.info
Acreditación Institucional de Alta Calidad Resolución 4792 del 15 de mayo de 2019
UNIVERSIDAD DE MANIZALES

- **grupo_academico.py (Clase GrupoAcademico):** Articula la oferta de una Asignatura orientada por un Docente en un Horario determinado, controlando la capacidad máxima y la lista de objetos Estudiante matriculados mediante mostrar_informacion_grupo(), matricular_estudiante(), retirar_estudiante() y mostrar_estudiantes().
- **excepciones.py:** Define cinco clases de excepción propias del dominio académico que heredan de Exception.
- **main.py:** Programa principal que ejecuta tanto los escenarios heredados de la Actividad Colaborativa N°1 como todas las pruebas mínimas de funcionamiento y manejo de excepciones de la Actividad Colaborativa N°2. Haciendo que el nuevo sistema 2.0 de Gestión Académica funcione con base en los lineamientos establecidos.

3. Desarrollo y Justificación de las Decisiones de Diseño

3.1. Implementación de Persona como Clase Abstracta y Comportamiento de las Subclases

En el dominio de una institución universitaria, tenemos claro que toda persona vinculada posee datos básicos de identificación (documento, nombre completo y correo institucional), pero ninguna persona se registra en la universidad bajo un rol genérico o indeterminado: necesariamente actúa como estudiante, docente o personal administrativo. En la primera versión del proyecto, la clase Persona podía ser instanciada directamente, lo que permitía crear objetos conceptualmente incompletos y obligaba a lanzar manualmente un NotImplementedError en el método realizar_actividad_principal().

Para representar fielmente la realidad del sistema en el archivo persona.py, se decidió transformar la clase Persona en una clase abstracta, haciéndola heredar de ABC (del módulo estándar abc de Python) y decorando el método realizar_actividad_principal(self) con @abstractmethod. Esta decisión técnica aporta dos beneficios clave, que son:

1. Imposibilidad de instanciación directa: Python impide crear objetos directamente de la clase Persona. Como se demuestra al inicio de main.py, el intento de ejecutar Persona('0000', 'Usuario Genérico', 'usuario@umanizales.edu.co') genera inmediatamente una excepción TypeError controlada mediante un bloque try/except, garantizando que en memoria solo existan roles concretos.

2. Contrato obligatorio para las clases derivadas: El decorador @abstractmethod obliga a que Estudiante, Docente y Administrativo implementen realizar_actividad_principal(). Cada subclase cumple este contrato aportando un comportamiento específico según sus atributos particulares: en Estudiante (estudiante.py), el método consulta self.__programa.get_nombre() y self.__semestre para informar el programa académico que cursa y su semestre actual; en Docente (docente.py), utiliza self.__facultad para indicar la facultad donde orienta clases; y en Administrativo (administrativo.py), emplea self.__cargo y self.__dependencia para detallar el cargo que desempeña y la dependencia administrativa a la que pertenece.

VIGILADO Por el Ministerio de Educación Nacional
Campusvirtual Universidad de Manizales
ISO 9001:2008 MSC-57117129 CERTIFIED www.fbcert.services www.certidata.info
Acreditación Institucional de Alta Calidad Resolución 4792 del 15 de mayo de 2019
UNIVERSIDAD DE MANIZALES

3.2. Conservación del Polimorfismo de la Actividad Colaborativa N°1

La transformación de Persona en una clase abstracta mantiene intacto y fortalece el polimorfismo implementado en la Actividad Colaborativa N°1. Dado que Persona define la interfaz común (mostrar_informacion() y realizar_actividad_principal()) y que las tres subclases mantienen su relación de herencia, el programa principal puede procesar cualquier subtipo de Persona bajo una misma referencia.

En main.py se conserva la lista heterogénea llamada personas, conformada por objetos de tipo Estudiante, Docente y Administrativo. Esta colección es recorrida mediante un único ciclo for persona in personas:, invocando directamente persona.mostrar_informacion() y persona.realizar_actividad_principal() sin utilizar condicionales por tipo (if/elif ni isinstance). Python resuelve dinámicamente en tiempo de ejecución qué versión sobrescrita del método debe ejecutar según la clase concreta de cada instancia.

3.3. Relaciones entre Grupo Académico, Asignatura, Docente, Estudiante y Horario

Para organizar la oferta académica sin caer en la mala práctica de almacenar textos aislados (como guardar únicamente el nombre de la asignatura o del profesor como cadenas de caracteres), la clase GrupoAcademico (grupo_academico.py) articula objetos reales previamente instanciados mediante relaciones bien diferenciadas:

• Relación entre GrupoAcademico y Asignatura (Asociación): El constructor de GrupoAcademico recibe un objeto de la clase Asignatura (validado con isinstance(asignatura, Asignatura)) y lo almacena en self.__asignatura. Así, el grupo accede en todo momento a los métodos del objeto Asignatura, como get_nombre() y get_codigo(), en mostrar_informacion_grupo().

• Relación entre GrupoAcademico y Docente (Asociación): El constructor recibe una instancia de Docente (validada con isinstance(docente, Docente)) en el atributo self.__docente, permitiendo consultar directamente los datos del profesor responsable del curso.

• Relación entre GrupoAcademico y Estudiante (Agregación / Matrícula): Cada grupo administra una colección privada self.__estudiantes_matriculados inicializada como lista vacía ([]). El método matricular_estudiante(estudiante) verifica que el argumento sea instancia de Estudiante y lo incorpora a la lista, mientras que retirar_estudiante(codigo_estudiantil) lo remueve. Se trata de una relación de agregación porque los objetos Estudiante tienen un ciclo de vida independiente del grupo: un mismo estudiante puede matricularse en varios grupos simultáneamente (por ejemplo, estudiante1 en GRP-POO-01 y GRP-BD-01) y sigue existiendo en el sistema aunque se retire de un grupo.

• Relación entre GrupoAcademico y Horario (Composición): Cada grupo contiene un objeto de la clase Horario (horario.py) en el atributo privado self.__horario, el cual

VIGILADO Por el Ministerio de Educación Nacional
Campusvirtual Universidad de Manizales
ISO 9001:2008 MSC-57117129 CERTIFIED www.fbcert.services www.certidata.info
Acreditación Institucional de Alta Calidad Resolución 4792 del 15 de mayo de 2019
UNIVERSIDAD DE MANIZALES

estructura el día (__dia), la hora de inicio (__hora_inicio), la hora de finalización (__hora_finalizacion) y el salón (__salon).

3.4. Por qué la Relación entre Grupo Académico y Horario es Composición y no Herencia

Una decisión completa en el diseño de la Fase 3 consistió en modelar el vínculo entre GrupoAcademico y Horario mediante una relación de composición ('tiene un' / parte-todo) en lugar de utilizar herencia ('es un') o una cadena de texto simple:

En primer lugar, el horario no se representó como un texto plano porque requiere encapsular y validar de forma independiente cuatro atributos propios (día, hora de inicio, hora de finalización y salón) mediante sus respectivos métodos get y set en horario.py. En segundo lugar, en el constructor de GrupoAcademico se instancia y encapsula internamente un objeto Horario propio (self.__horario = Horario(...)) a partir de los datos suministrados, de manera que dicho objeto Horario se crea como parte constitutiva de la organización del grupo y describe cuándo y dónde se desarrolla.

Esta relación no corresponde a herencia porque la herencia solo aplica cuando una clase es una especialización taxonómica de otra (por ejemplo, Estudiante 'es una' Persona). Semánticamente y estructuralmente, un GrupoAcadémico NO 'es un' Horario, ni un Horario 'es un' GrupoAcadémico: el grupo académico es una unidad de oferta educativa que 'tiene un' horario asignado. Aplicar herencia entre ambos conceptos violaría el principio de sustitución y mezclaría las responsabilidades de gestión de cupos y matrículas con los atributos de programación horaria.

3.5. Control de Situaciones Excepcionales mediante Excepciones Personalizadas

Para controlar las situaciones anómalas durante la matrícula y la consulta de información sin detener abruptamente el programa ni limitarse a condicionales informativos, se creó el módulo excepciones.py con cinco clases propias que heredan de Exception. En la siguiente tabla se detalla qué situación controla cada una y en qué método del sistema se genera:

Excepción Personalizada	Dónde se genera	Situación que controla
EstudianteYaMatriculadoError	GrupoAcademico.matricular_estudiante() en grupo_academico.py	Detecta si el código estudiantil del alumno que se intenta inscribir ya se encuentra registrado en self.__estudiantes_matriculados, impidiendo matrículas duplicadas en el mismo grupo.
CapacidadMaximaAlcanzadaError	GrupoAcademico.matricular_estudiante() en grupo_academico.py	Se activa cuando la cantidad actual de inscritos (len(self.__estudiantes_matriculados)) ya iguala la capacidad máxima

VIGILADO Por el Ministerio de Educación Nacional
Campusvirtual Universidad de Manizales
ISO 9001:2008 MSC-57117129 www.fbcert.services www.certidata.info
Acreditación Institucional de Alta Calidad Resolución 4792 del 15 de mayo de 2019
UNIVERSIDAD DE MANIZALES

		(__capacidad_maxima) del grupo.
EstudianteNoMatriculadoError	GrupoAcademico.retirar_estudiante() en grupo_academico.py	Se lanza cuando se solicita retirar un código estudiantil que no figura dentro de la lista de estudiantes matriculados del grupo.
AsignaturaNoEncontradaError	Asignatura.buscar_asignatura() en asignatura.py	Se lanza al recorrer la lista de asignaturas del sistema cuando ningún objeto coincide con el código de asignatura buscado.
ProgramaAcademicoNoEncontradoError	ProgramaAcademico.buscar_programa() en programa_academico.py	Se lanza al recorrer la lista de programas académicos cuando ningún objeto coincide con el código de programa consultado.

En main.py, cada uno de estos escenarios de prueba se invoca dentro de su respectivo bloque try / except. Cuando el método de la clase detecta la condición inválida y ejecuta la sentencia raise con la excepción personalizada, el bloque except en main.py la captura, muestra el mensaje comprensible al usuario y permite que el flujo del programa continúe ejecutando las pruebas restantes hasta finalizar con normalidad.

3.6. Conservación del Polimorfismo de la Actividad Colaborativa N°1 en la Nueva Versión del Sistema

El polimorfismo desarrollado en la Actividad Colaborativa N°1 no solo se conserva íntegramente en esta segunda versión del sistema, sino que la hemos fortalecido gracias a la conversión de Persona en una clase abstracta (class Persona(ABC) en persona.py).

En la primera entrega, el polimorfismo se apoyaba en que Estudiante, Docente y Administrativo heredaban de Persona y sobrescribían los métodos mostrar_informacion() y realizar_actividad_principal(). Sin embargo, no existía una restricción formal del lenguaje que impidiera crear una subclase sin implementar dicho comportamiento. En la nueva versión del sistema, al declarar realizar_actividad_principal() con el decorador @abstractmethod dentro de Persona, se establece un contrato polimórfico estricto: Python garantiza que toda subclase concreta que herede de Persona posea obligatoriamente su propia implementación de este método antes de poder ser instanciada.

En la práctica, este polimorfismo se mantiene y demuestra dentro del archivo main.py de tres maneras complementarias:

1. **Colección heterogénea bajo una interfaz común:**
   En main.py se define una única lista denominada personas, en la cual se almacenan conjuntamente objetos de distintas clases derivadas (estudiante1, estudiante2, estudiante3 de tipo Estudiante; docente1, docente2 de tipo Docente; y administrativo1, administrativo2 de tipo Administrativo). Todos ellos pueden

VIGILADO Por el Ministerio de Educación Nacional
Campusvirtual Universidad de Manizales
ISO 9001:2008 MSC-57117129 www.fbcert.services www.certidata.info
Acreditación Institucional de Alta Calidad Resolución 4192 del 15 de mayo de 2019
UNIVERSIDAD DE MANIZALES

coexistir y ser tratados uniformemente porque comparten la superclase abstracta Persona.

2. **Recorrido con un único ciclo sin condicionales por tipo:** La lista de personas se recorre mediante una sola instrucción iterativa (for persona in personas:) que invoca directamente persona.mostrar_informacion() y persona.realizar_actividad_principal(), sin utilizar estructuras condicionales (if/elif) ni verificaciones de tipo (isinstance() o type()) para distinguir si el objeto actual es un estudiante, un docente o un administrativo.
3. **Enlace dinámico (*Dynamic Binding*) y sobrescritura diferenciada:** Durante la ejecución del ciclo for, Python determina automáticamente qué versión del método ejecutar según la clase real de cada objeto en memoria:
    - En **mostrar_informacion()**, cada subclase reutiliza primero la lógica común llamando a super().mostrar_informacion() (que imprime identificación, nombre y correo desde Persona) y acto seguido imprime sus atributos específicos (código estudiantil, programa, semestre y promedio en Estudiante; número de empleado, facultad, contratación y horas en Docente; número de empleado, dependencia, cargo y jornada en Administrativo).
    - En **realizar_actividad_principal()**, cada objeto responde con una acción distinta acorde a su rol: Estudiante informa el nombre de su ProgramaAcademico y el semestre que cursa; Docente informa la facultad en la que orienta clases; y Administrativo informa el cargo que desempeña y su dependencia.

De igual forma, los mismos objetos estudiante1, estudiante2, estudiante3 y docente1 que participan en la demostración polimórfica de personas son reutilizados posteriormente en main.py para conformar las instancias de GrupoAcademico (grupo_poo_01 y grupo_bd_01), demostrando que la evolución hacia grupos, horarios y matrículas se integra sobre las mismas entidades sin romper la arquitectura polimórfica original.

4. Conclusiones

1. La implementación de Persona como clase abstracta mediante ABC y @abstractmethod fortaleció el modelo del dominio, impidiendo la creación de personas sin rol definido y asegurando que Estudiante, Docente y Administrativo implementen obligatoriamente realizar_actividad_principal().
2. El diseño de GrupoAcademico y Horario permitió aplicar y diferenciar en la práctica las relaciones de asociación, agregación y composición frente a la herencia, garantizando que el grupo opere sobre objetos reales de Asignatura, Docente, Estudiante y Horario.
3. La creación de cinco excepciones personalizadas en excepciones.py desacopló la detección de errores de negocio (mediante raise en los métodos de las clases) de su tratamiento en el programa principal (mediante bloques try/except en main.py), asegurando que el sistema continúe su ejecución ante cualquier fallo controlado.

VIGILADO Por el Ministerio de Educación Nacional
Campusvirtual Universidad de Manizales
CERTIFIED ISO 9001:2008 MSC-57117129 www.fbcert.services www.certidata.info
Acreditación Institucional de Alta Calidad Resolución 4792 del 15 de mayo de 2019
UNIVERSIDAD DE MANIZALES

4. La evolución sobre el mismo proyecto de la Actividad Colaborativa N°1 demostró la mantenibilidad y extensibilidad de la Programación Orientada a Objetos, conservando el encapsulamiento y el polimorfismo original mientras se integraron los nuevos procesos de oferta y matrícula.

5. Declaración de Uso de Herramientas de Inteligencia Artificial

De acuerdo con los lineamientos establecidos en la actividad, los integrantes del equipo declaran el uso de una herramienta de inteligencia artificial como apoyo para la revisión de código, validación de buenas prácticas en la implementación de clases abstractas, composición y excepciones personalizadas en Python. Los integrantes del grupo revisaron, ejecutaron y comprendieron en su totalidad la sintaxis del código entregado, su funcionamiento y las decisiones de diseño aquí sustentadas.

VIGILADO Por el Ministerio de Educación Nacional
Campusvirtual Universidad de Manizales
ISO 9001:2008 MSC-57117129 CERTIFIED www.fbcert.services www.certidata.info
Acreditación Institucional de Alta Calidad Resolución 4752 del 15 de mayo de 2019
UNIVERSIDAD DE MANIZALES